GitHub Actions: Timezone Support for Scheduled Workflows

DXC Open Office Hour — GitHub Actions Feature Demo

Agenda

Segment	Time
🎯 Presentation	10 min
🛠️ Live Demo	10 min
❓ Open Q&A	30 min
📋 Crafted Q&A	10 min
Total	**60 min**

Section 1: Presentation (10 minutes)

Slide 1 — Title

⏰ GitHub Actions: Timezone Support

A FY2026 Feature Release

March 2026 • GitHub Actions Changelog

> **Speaker Notes:**
> Welcome everyone to today's GitHub Actions open office hour with DXC. In the next 20 minutes, we'll explore a newly released feature that makes scheduled workflows much more intuitive — timezone support for `schedule` triggers. Then we'll open the floor for Q&A. Let's dive in.

Slide 2 — The Problem Before This Feature

📅 Scheduled Workflows: The Old Way

Previously, all `schedule` triggers ran in UTC only.

```
on:
  schedule:
    - cron: '0 9 * * 1-5'   # What time is this for your team?
```

Pain Points

- ❌ Developers had to manually convert local time → UTC
- ❌ Daylight Saving Time (DST) shifts broke "9 AM" runs
- ❌ Multi-region teams debugged confusing pipeline fire times
- ❌ On-call engineers got paged at 3 AM because "9 AM UTC" wasn't what they intended

> **Speaker Notes:**
> Before this release, every scheduled workflow fired in UTC — full stop. If you wanted a nightly job to run at midnight Eastern Time, you had to mentally (or with a calculator) convert that to 5:00 UTC in winter and 4:00 UTC in summer. And if you forgot to account for Daylight Saving Time, your "nightly" job quietly started firing an hour off. This was a common source of confusion for teams spread across time zones.

Slide 3 — The New Feature: Timezone Support

✅ Scheduled Workflows: The New Way

GitHub Actions `schedule` triggers now accept a `timezone` field!

```
on:
  schedule:
    - cron: '0 9 * * 1-5'
      timezone: 'America/New_York'   # ← NEW in March 2026!
```

What Changed

- ✅ Specify any **IANA timezone name** (e.g. `America/Chicago`, `Europe/London`, `Asia/Tokyo`)
- ✅ GitHub Actions automatically handles **Daylight Saving Time**
- ✅ No more UTC arithmetic for your team
- ✅ Cron expression reads exactly as the team intends it

> **Speaker Notes:**
> In the March 2026 release, GitHub added a single, elegant field: `timezone`. You just add it under your `cron` entry, use any IANA timezone name, and GitHub handles all the UTC conversion and DST adjustments for you. The workflow YAML now reads exactly as your team intended — "run at 9 AM New York time, Monday through Friday."

Slide 4 — IANA Timezone Names

🌍 IANA Timezone Reference

Common timezones your team will use:

Region	IANA Timezone
US Eastern	`America/New_York`
US Central	`America/Chicago`
US Mountain	`America/Denver`
US Pacific	`America/Los_Angeles`
UK / Ireland	`Europe/London`
Central Europe	`Europe/Berlin` or `Europe/Paris`
India	`Asia/Kolkata`
Japan	`Asia/Tokyo`
Australia East	`Australia/Sydney`
UTC (no change)	`UTC`

Full list: [IANA Time Zone Database]

> **Speaker Notes:**
> The timezone values are standard IANA names — the same ones used in Linux systems, databases, and programming languages. You don't need to learn a new format. If your team already knows "America/New_York" from their cloud infrastructure configs, they already know how to use this feature.

Slide 5 — Real-World Use Cases

💼 Real-World Use Cases

Use Case 1: Business-Hours Nightly Build

```
# Run at 11:30 PM US Central — after business hours, before midnight
- cron: '30 23 * * 1-5'
  timezone: 'America/Chicago'
```

Use Case 2: Morning Health Check

```
# Ping production APIs every morning at 8 AM London time
- cron: '0 8 * * *'
  timezone: 'Europe/London'
```

Use Case 3: Global Team — Multiple Schedules

```
on:
  schedule:
    - cron: '0 9 * * 1-5'
      timezone: 'America/New_York'     # US team morning run
    - cron: '0 9 * * 1-5'
      timezone: 'Asia/Tokyo'           # APAC team morning run
```

> **Speaker Notes:**
> Here are three patterns that come up constantly. The first is a nightly build that needs to run after the US Central business day ends. The second is a morning health check aligned to a London-based SRE team. The third is a multi-region team that wants the same logical schedule — 9 AM on a Monday — to fire independently for each region's team. Multiple `cron` entries under `schedule` is supported, so you can fan these out cleanly.

Slide 6 — Key Takeaways

🚀 Key Takeaways

1. **One field, zero UTC math** — add `timezone:` under your cron entry
2. **DST is handled automatically** — no seasonal adjustments needed
3. **IANA standard** — same format used everywhere in your stack

4. **Multiple schedules per workflow** — mix timezones freely
5. **Backward compatible** — existing workflows without `timezone` continue to run in UTC

> **No migration required. Opt in when it helps your team.**

> **Speaker Notes:**
> The five things to remember: it's one field, DST is automatic, it's IANA standard, you can have multiple schedules per workflow, and it is fully backward compatible — nothing breaks if you do nothing. This is a purely additive feature. Encourage your teams to start adding timezones to new scheduled workflows and update existing ones when they cause confusion.

Section 2: Live Demo (10 minutes)

Demo Guide

> See `DEMO.md` for the full step-by-step walkthrough with screenshots guidance.

Demo Flow (10 minutes):

Step	Activity	Time
1	Open the repo and show the demo workflows in `.github/workflows/`	2 min
2	Walk through `demo-before-timezone.yml` — the old UTC-only approach	2 min
3	Walk through `demo-with-timezone.yml` — the new timezone-aware approach	2 min
4	Show `demo-multi-region.yml` — multiple region schedules	2 min
5	Trigger a workflow run manually and show the Actions tab output	2 min

---

Section 3: Open Q&A (30 minutes)

Q&A Facilitation Guide

> **Open the floor to the audience.**

Suggested opener:

> "We've got 30 minutes for open questions — anything about timezone support, scheduled workflows, or GitHub Actions broadly. Who wants to go first?"

Common themes to anticipate:

- "Can we use this in GitHub Enterprise Server?"
- "Does this work with `workflow_dispatch` triggers too?" *(No — only `schedule`)*
- "What happens if I specify an invalid timezone?"
- "Is there a way to see the next scheduled run time in the UI?"
- "Can we use `env` variables for the timezone?"

Facilitation tips:

- Repeat questions for the room before answering
- Park complex questions on a whiteboard/chat for follow-up
- Invite others to answer before you do

Section 4: Crafted Q&A (10 minutes)

Q1 — What's the minimal GitHub Actions workflow we should start with?

Q: *"What's the minimal GitHub Actions workflow we should start with?"*

A: Start with a single workflow that runs on `push` and `pull_request`, with one job that checks out code and runs tests. Keep it simple, then add build/package/deploy stages once the team trusts the basics.

```yaml
# .github/workflows/ci.yml
name: CI

on:
  push:
  pull_request:

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Run tests
        run: npm test   # or your equivalent
```

> **Mental model:** triggers → jobs → steps → actions

Q2 — Where should workflow YAML live, and how do we organize it?

Q: *"Where should workflow YAML live, and how do we organize it?"*

A: Put YAML in `.github/workflows/`. Use **one workflow per purpose** (CI, nightly, release, deploy), and name them clearly. For larger repos, factor repeated logic into reusable workflows or composite actions.

```
.github/
  workflows/
    ci.yml          ← on push / pull_request
    nightly.yml     ← scheduled build
    release.yml     ← on tag push
    deploy.yml      ← on workflow_dispatch or merge to main
```

> As the repo grows, keep the file names self-documenting. Avoid one giant `main.yml` that does everything.

Q3 — How do we keep workflows maintainable as they grow?

Q: *"How do we keep workflows maintainable as they grow?"*

A: Use three patterns:

1. Reusable workflows — for shared pipelines across org repos

```
# Call a centrally-maintained workflow
jobs:
  call-central-ci:
    uses: my-org/.github/.github/workflows/standard-ci.yml@main
```

2. Composite actions — for repeated step blocks (lint/test/setup)

```
# .github/actions/setup-node/action.yml
runs:
  using: composite
  steps:
    - uses: actions/setup-node@v4
    - run: npm ci
      shell: bash
```

3. Matrix builds — for variations (OS, language versions) without duplicating YAML

```
strategy:
  matrix:
    os: [ubuntu-latest, windows-latest, macos-latest]
    node: [18, 20, 22]
```

Q4 — What's the difference between an Action and a workflow?

Q: *"What's the difference between an Action and a workflow?"*

A:

Concept	What it is	Where it lives
Workflow	The YAML pipeline definition	`.github/workflows/*.yml`
Action	A reusable unit called inside steps	Marketplace, a repo, or local path

```
jobs:
  build:
    steps:
      - uses: actions/checkout@v4        # ← this is an Action
      - uses: actions/setup-node@v4      # ← this is an Action
      - run: npm test                    # ← this is a step (shell command)
```

> **One-liner:** Workflows *orchestrate*; actions *do work*.

Q5 — How do we stop long queues / slow pipelines?

Q: *"How do we stop long queues / slow pipelines?"*

A: Tackle it in this order:

1. Right-size job parallelism — split independent jobs so they run concurrently

```
jobs:
  lint:    { ... }   # runs in parallel
  test:    { ... }   # runs in parallel
  build:             # runs after lint + test
    needs: [lint, test]
```

2. Use caching — cache dependencies to avoid reinstalling on every run

```
- uses: actions/cache@v4
  with:
    path: ~/.npm
    key: ${{ runner.os }}-npm-${{ hashFiles('**/package-lock.json') }}
```

3. Avoid rebuilding — pass compiled artifacts between jobs using `upload-artifact` / `download-artifact` instead of rebuilding in each job

---

Presentation prepared for DXC GitHub Actions Open Office Hour
Feature reference: [GitHub Actions Late March 2026 Updates]